বাংলাদেশ আর্মি ইন্টারন্যাশনাল ইউনিভার্সিটি অব সায়েন্স এন্ড টেকনোলজি, কুমিল্লা
BANGLADESH ARMY INTERNATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY (BAIUST), CUMILLA
Department of Computer Science and Engineering

Lab Report

Lab Report No	:	**02**
Lab Report Name	:	**Solution of 8-Puzzle Problem using DFS, DLS and IDS**
Course Title	:	Artificial Intelligence Sessional
Course Code	:	CSE 422
Name	:	MD Rakibul Hassan
ID	:	0822220105101024
Level : **4**	**Term** : **2**	**Section** : **A** **Group** : **G1**
Date of Submission	:	**10/05/2026** **Semester** : Spring **Year** : 2026

Marking Rubric

Problem Understanding & Report Clarity (3)	Implementation (4)	Results, Analysis & Presentation (3)	Total (10)

Key Learnings:

- Learned about the DFS, DLS and IDS algorithms with hands-on experience.
- Understand the working of the usage limit and the repeatedly applying DLS by increasing the depth limit step by step until the goal is not found.

Code Implementation:

Using DFS

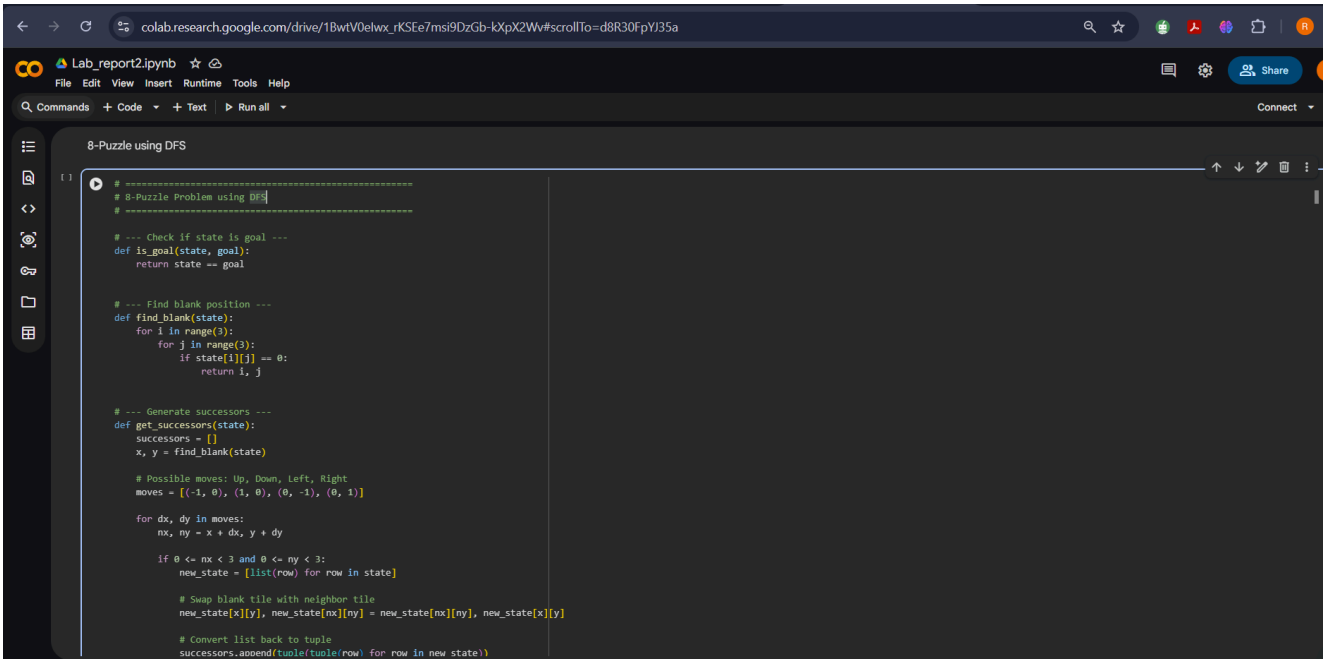
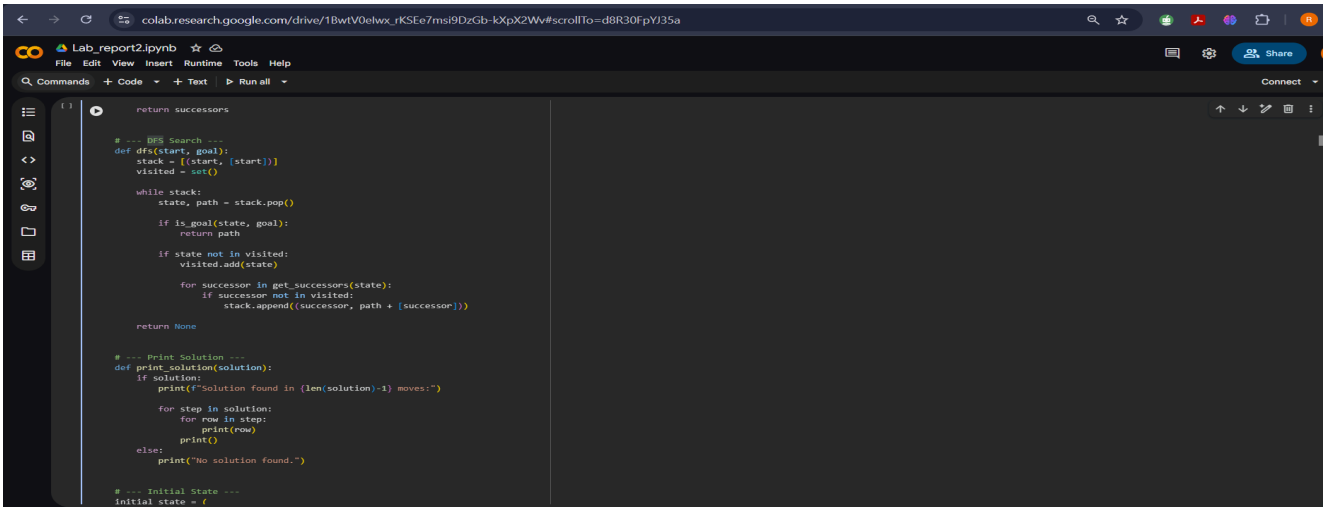
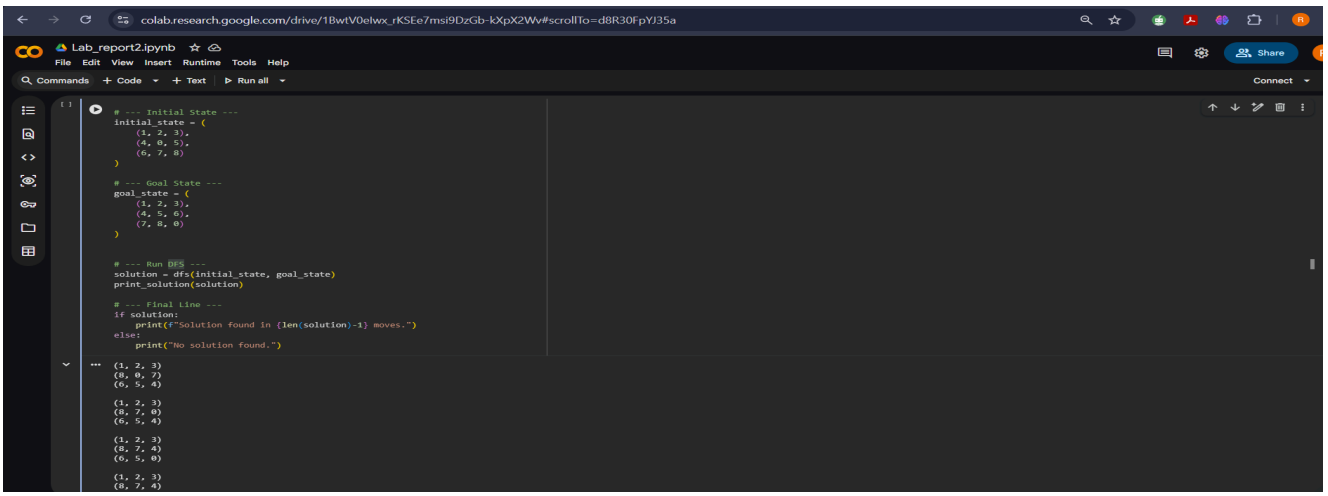


Figure No: 01



```
    return successors


# --- DFS Search ---
def dfs(start, goal):
    stack = [(start, [start])]
    visited = set()

    while stack:
        state, path = stack.pop()

        if is_goal(state, goal):
            return path

        if state not in visited:
            visited.add(state)

            for successor in get_successors(state):
                if successor not in visited:
                    stack.append((successor, path + [successor]))

    return None


# --- Print Solution ---
def print_solution(solution):
    if solution:
        print(f"Solution found in {len(solution)-1} moves:")

        for step in solution:
            for row in step:
                print(row)
            print()
    else:
        print("No solution found.")


# --- Initial State ---
initial_state = (
```

Figure No: 02



```
# --- Initial State ---
initial_state = (
    (1, 2, 3),
    (4, 0, 5),
    (6, 7, 8)
)

# --- Goal State ---
goal_state = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)


# --- Run DFS ---
solution = dfs(initial_state, goal_state)
print_solution(solution)

# --- Final Line ---
if solution:
    print(f"Solution found in {len(solution)-1} moves.")
else:
    print("No solution found.")
```

```
(1, 2, 3)
(8, 0, 7)
(6, 5, 4)

(1, 2, 3)
(8, 7, 0)
(6, 5, 4)

(1, 2, 3)
(8, 7, 4)
(6, 5, 0)

(1, 2, 3)
(8, 7, 4)
```

Figure No: 03

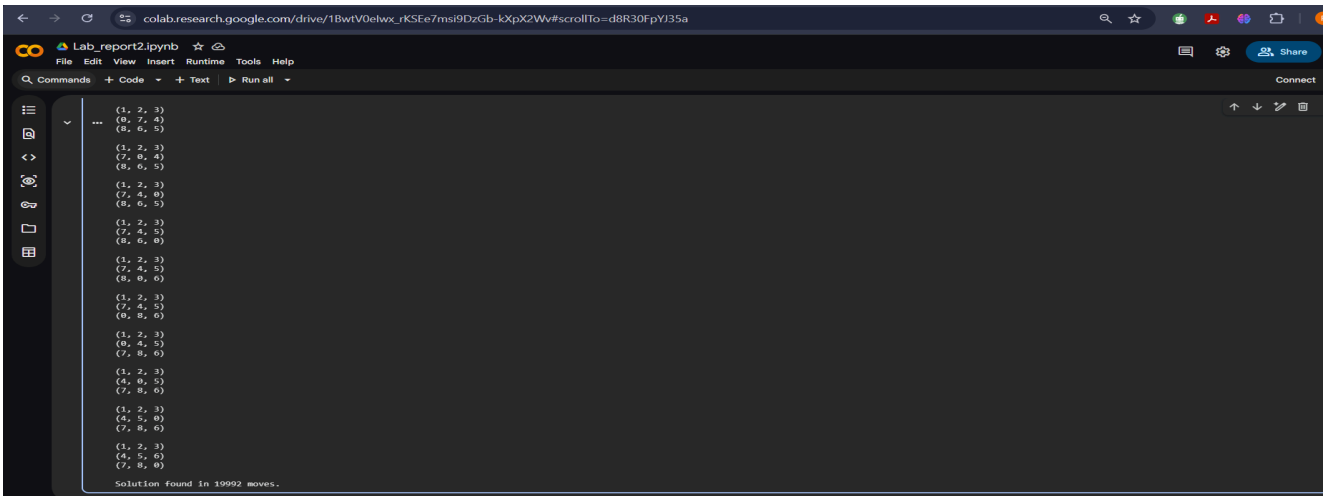
Sample Input, Output, and Result Analysis:

Input Sample:

initial_state = ((1, 2, 3), (4, 0, 5), (6, 7, 8))

goal_state = ((1, 2, 3), (4, 5, 6), (7, 8, 0))

Output Sample:



```
(1, 2, 3)
(0, 7, 4)
(8, 6, 5)

(1, 2, 3)
(7, 0, 4)
(8, 6, 5)

(1, 2, 3)
(7, 4, 0)
(8, 6, 5)

(1, 2, 3)
(7, 4, 5)
(8, 6, 0)

(1, 2, 3)
(7, 4, 5)
(8, 0, 6)

(1, 2, 3)
(7, 4, 5)
(0, 8, 6)

(1, 2, 3)
(0, 4, 5)
(7, 8, 6)

(1, 2, 3)
(4, 0, 5)
(7, 8, 6)

(1, 2, 3)
(4, 5, 0)
(7, 8, 6)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Solution found in 19992 moves.
```

Figure No: 04

Using DLS

Figure No: 05

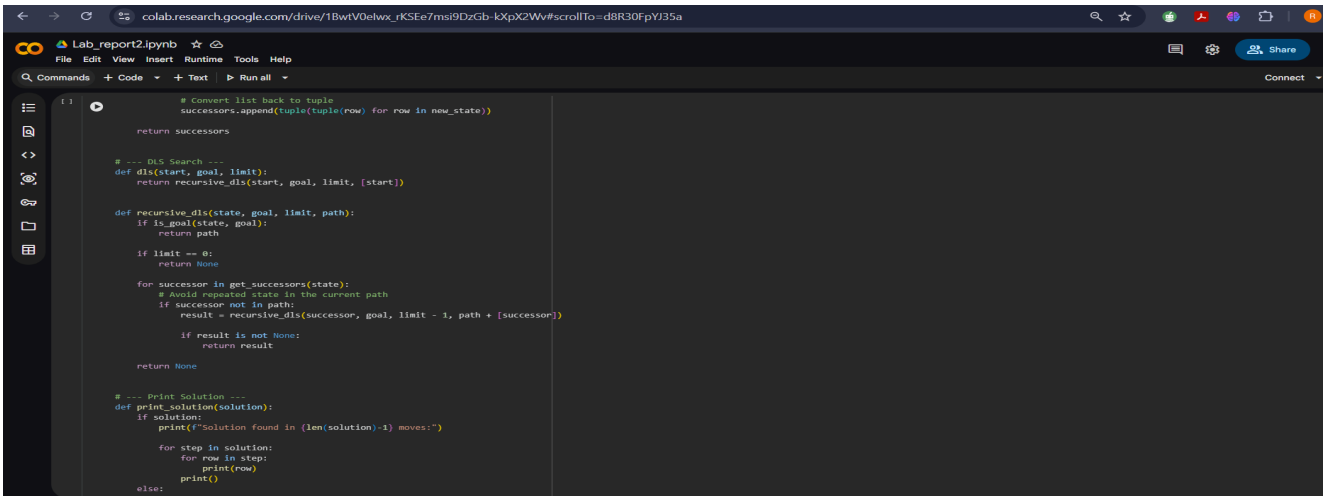


Figure No: 06

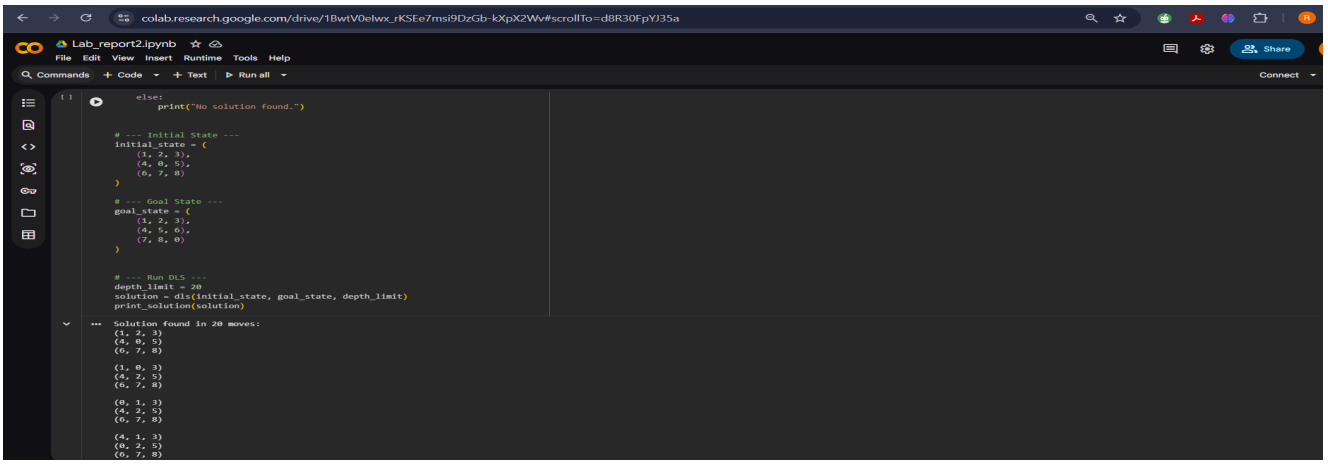


Figure No: 07

Sample Input, Output, and Result Analysis:

Input Sample:

initial_state = ((1, 2, 3), (4, 0, 5), (6, 7, 8))

goal_state = ((1, 2, 3), (4, 5, 6), (7, 8, 0))

Output Sample:

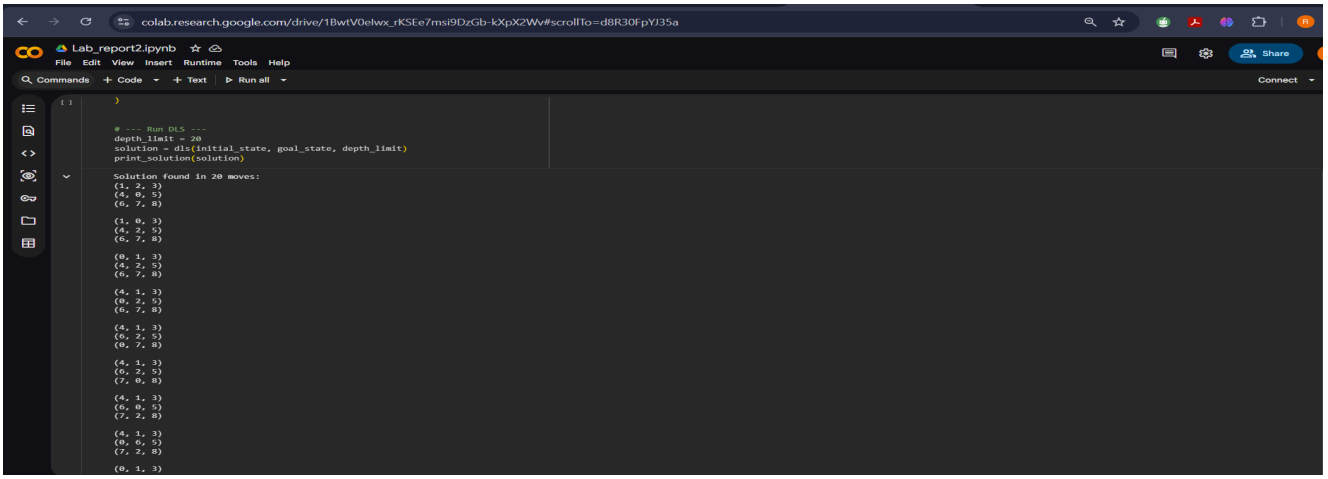


Figure No: 08

Using IDS

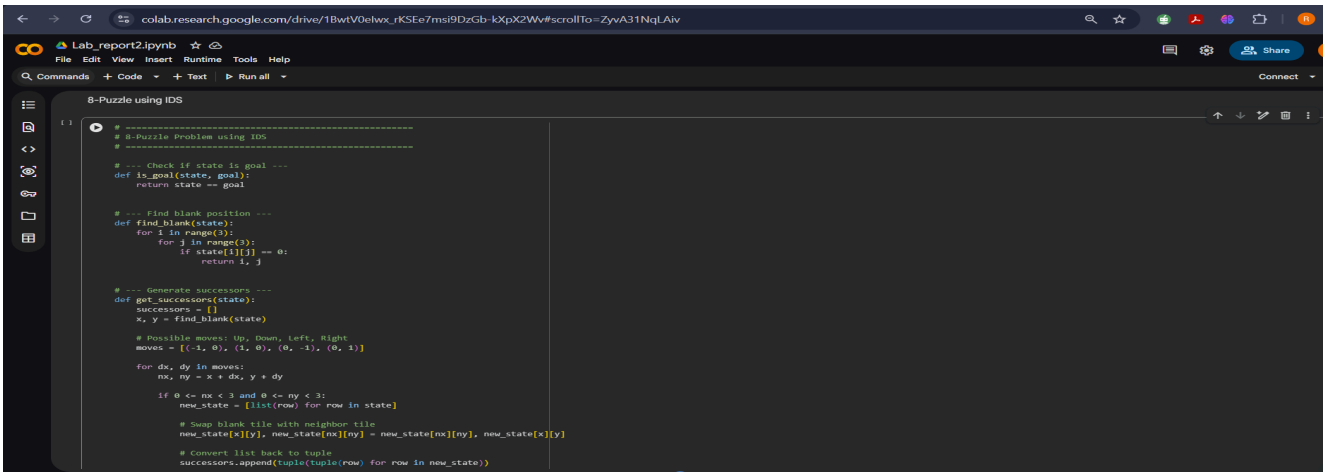


Figure No: 09

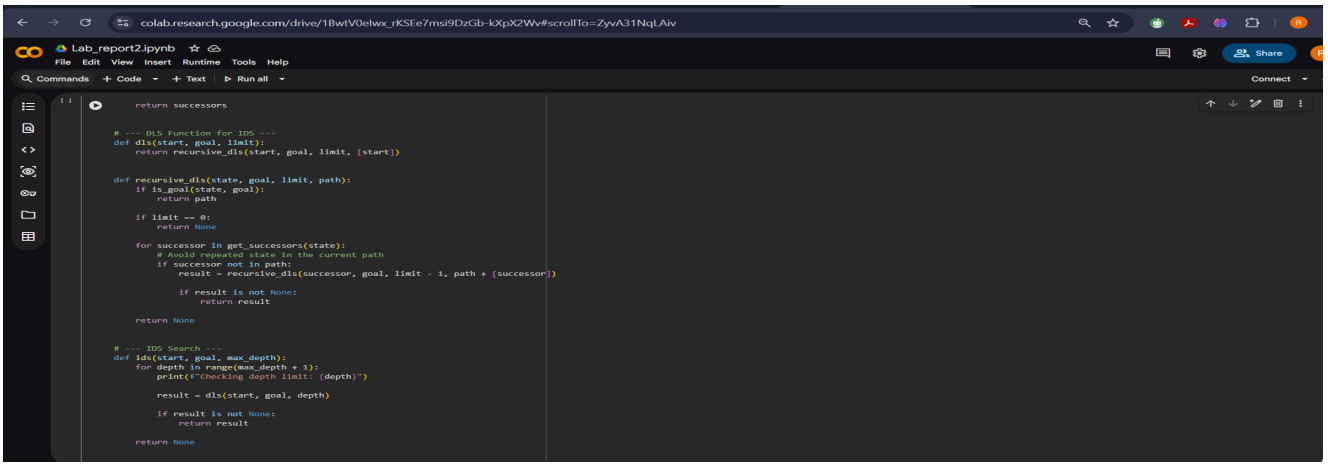


Figure No: 10

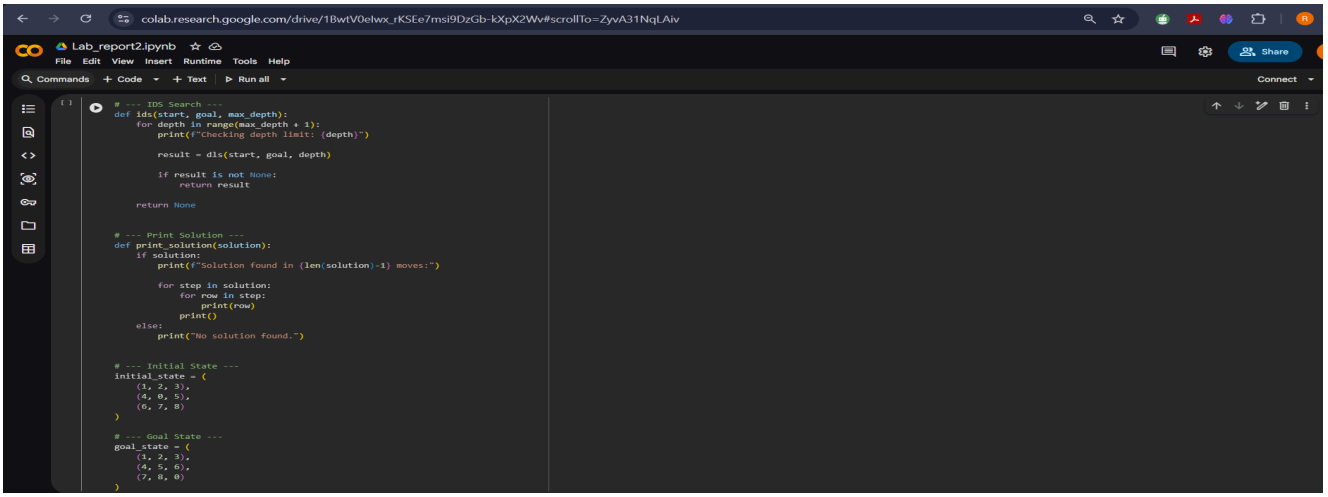


Figure No: 11

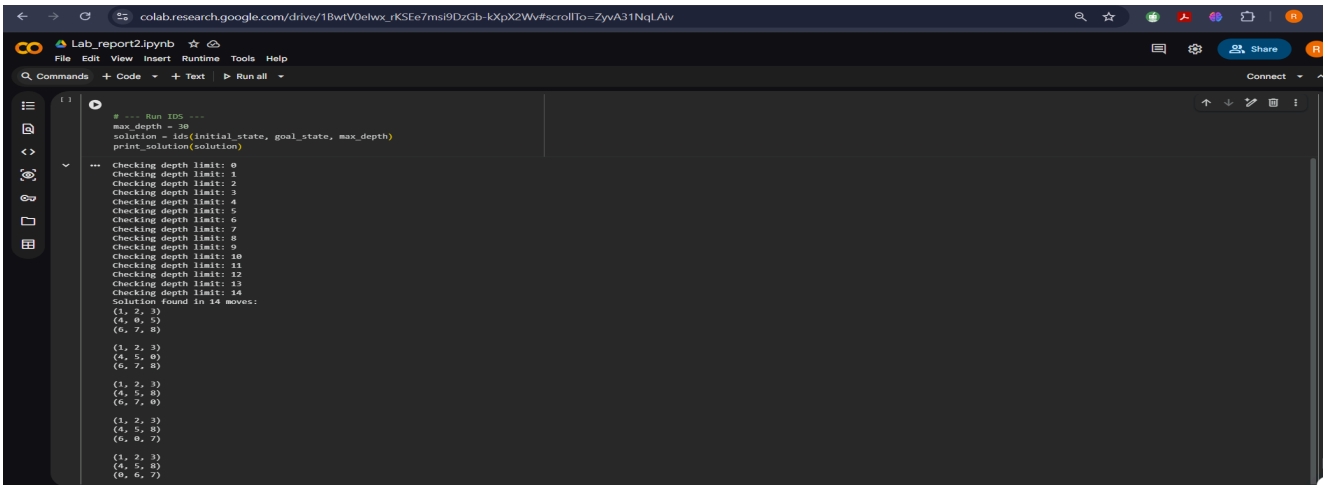


Figure No: 12

Sample Input, Output, and Result Analysis:

Input Sample:

initial_state = ((1, 2, 3), (4, 0, 5), (6, 7, 8))

goal_state = ((1, 2, 3), (4, 5, 6), (7, 8, 0))

Output Sample:

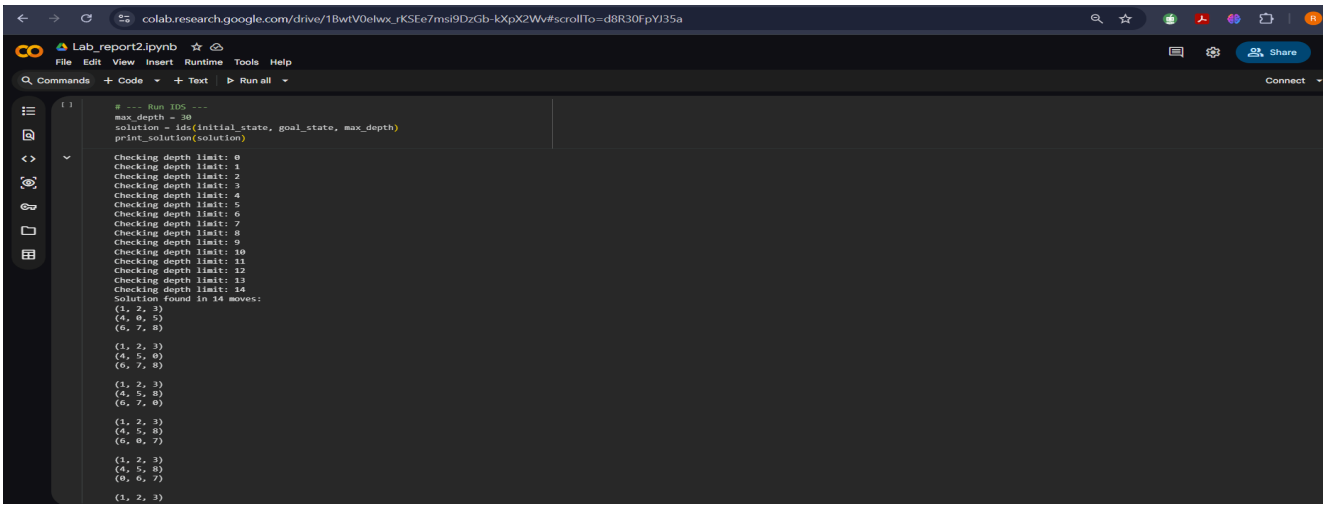


Figure No: 13

Result Analysis

- **While DFS can solve the problem, BFS offers a more optimal solution for it.**
- **A total of 19,992 moves using DFS, 20 moves using DLS and 14 moves in IDS are needed for solving the 8-puzzle problem.**
- **DFS uses a stack or recursion.**
- **DLS also uses a stack or recursion but stops exploration once it reaches a pre-defined depth limit.**
- **IDS repeatedly uses a stack or recursion to perform DLS at increasingly deeper limits until the goal is found.**